

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Analytical Problem Solving

Code of Conduct:

*I Nirmal Kumar T/192412003 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Nirmal Kumar T

Annexure A

Short Answer Test

1, A simple perceptron model is used to classify whether a student passes or fails based on two input features, namely study hours and attendance. The weights associated with these inputs are 0.6 and 0.4 respectively, and the bias value is -0.5 . For a given student with study hours equal to 2 and attendance equal to 1, compute the net input to the perceptron and apply the step activation function to determine the final output. Based on the result, conclude whether the student passes or fails. (5 Marks)

Given:

- Study Hours (x_1) = 2
- Attendance (x_2) = 1
- Weight for Study Hours (w_1) = 0.6
- Weight for Attendance (w_2) = 0.4
- Bias (b) = -0.5

Formula:

$$\text{Net Input} = (w_1 \times x_1) + (w_2 \times x_2) + b$$

Calculation:

$$\begin{aligned}\text{Net Input} &= (0.6 \times 2) + (0.4 \times 1) + (-0.5) \\ &= 1.2 + 0.4 - 0.5 \\ &= \mathbf{1.1}\end{aligned}$$

Step Activation Function:

- If Net Input ≥ 0 , Output = 1
- If Net Input < 0 , Output = 0

Since the Net Input is **1.1**, which is greater than 0,

Output = 1

Conclusion:

The perceptron output is **1**, which indicates that the student **Passes**.

Final Result:

- Net Input = **1.1**
- Perceptron Output = **1**
- Student Status = **Pass**

Python Program

Perceptron Model for Student Pass/Fail Classification

Input values

study_hours = 2

attendance = 1

Weights

w1 = 0.6

w2 = 0.4

Bias

bias = -0.5

Calculate net input

net_input = (w1 * study_hours) + (w2 * attendance) + bias

Step activation function

if net_input >= 0:

output = 1

result = "Pass"

else:

output = 0

result = "Fail"

Display the results

print("Net Input =", net_input)

print("Perceptron Output =", output)

print("Student Result =", result)

Output:

```
# Perceptron Model for Student Pass/Fail Classification

# Input values
study_hours = 2
attendance = 1

# Weights
w1 = 0.6
w2 = 0.4

# Bias
bias = -0.5

# Calculate net input
net_input = (w1 * study_hours) + (w2 * attendance) + bias

# Step activation function
if net_input >= 0:
    output = 1
    result = "Pass"
else:
    output = 0
    result = "Fail"

# Display the results
print("Net Input =", net_input)
print("Perceptron Output =", output)
print("Student Result =", result)

Net Input = 1.1
Perceptron Output = 1
Student Result = Pass
```

2. In a neural network, a single neuron is trained using the backpropagation learning rule. The initial weight is 0.5 and the learning rate is 0.1. For an input value of 2, the actual output produced by the neuron is 0.6, whereas the target output is 1. Calculate the error in the output and determine the weight update using the delta rule. Hence, compute the new updated weight after one iteration of training. (5 Marks)

Given:

- Initial Weight (w) = 0.5
- Learning Rate (η) = 0.1
- Input (x) = 2
- Actual Output (y) = 0.6
- Target Output (t) = 1

Formula 1: Calculate the Error

$$\text{Error} = \text{Target Output} - \text{Actual Output}$$

Calculation:

$$\begin{aligned}\text{Error} &= 1 - 0.6 \\ &= 0.4\end{aligned}$$

Formula 2: Delta Rule (Weight Update)

$$\Delta w = \eta \times \text{Error} \times \text{Input}$$

Where:

- η = Learning Rate
- Error = Target Output – Actual Output
- Input = x

Calculation:

$$\begin{aligned}\Delta w &= 0.1 \times 0.4 \times 2 \\ &= 0.08\end{aligned}$$

Formula 3: Updated Weight

$$\text{New Weight} = \text{Old Weight} + \Delta w$$

Calculation:

$$\begin{aligned}\text{New Weight} &= 0.5 + 0.08 \\ &= 0.58\end{aligned}$$

Conclusion:

After one iteration of backpropagation learning:

- Error = **0.4**
- Weight Update (Δw) = **0.08**
- Updated Weight = **0.58**

Hence, the new weight after one training iteration is **0.58**.

Backpropagation Weight Update using Delta Rule

Output:

```
weight = 0.5
learning_rate = 0.1
input_value = 2
target_output = 1
actual_output = 0.6

# Calculate error
error = target_output - actual_output

# Calculate weight update
delta_w = learning_rate * error * input_value

# Update weight
new_weight = weight + delta_w

# Display results
print("Error =", error)
print("Weight Update =", delta_w)
print("Updated Weight =", new_weight)
```

```
Error = 0.4
Weight Update = 0.080000000000000002
Updated Weight = 0.58000000000000001
```

3. A deep learning model is developed for disease prediction and its performance is evaluated using a confusion matrix. The model yields 180 true positives, 150 true negatives, 20 false positives, and 50 false negatives. Using these values, compute the accuracy, precision, sensitivity (recall), and specificity of the model. Based on the calculated results, discuss whether the model is reliable for medical diagnosis, considering the importance of minimizing diagnostic errors. (5 Marks)

Given:

- True Positives (TP) = 180
- True Negatives (TN) = 150
- False Positives (FP) = 20
- False Negatives (FN) = 50

Formula 1: Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Calculation:

$$\begin{aligned}\text{Accuracy} &= (180 + 150) / (180 + 150 + 20 + 50) \\ &= 330 / 400 \\ &= \mathbf{0.825}\end{aligned}$$

$$\text{Accuracy} = \mathbf{82.5\%}$$

Formula 2: Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Calculation:

$$\begin{aligned}\text{Precision} &= 180 / (180 + 20) \\ &= 180 / 200 \\ &= \mathbf{0.90}\end{aligned}$$

$$\text{Precision} = \mathbf{90\%}$$

Formula 3: Sensitivity (Recall)

$$\text{Sensitivity (Recall)} = \text{TP} / (\text{TP} + \text{FN})$$

Calculation:

$$\begin{aligned}\text{Sensitivity} &= 180 / (180 + 50) \\ &= 180 / 230 \\ &= \mathbf{0.7826}\end{aligned}$$

$$\text{Sensitivity} = \mathbf{78.26\%}$$

Formula 4: Specificity

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

Calculation:

$$\begin{aligned}\text{Specificity} &= 150 / (150 + 20) \\ &= 150 / 170 \\ &= \mathbf{0.8824}\end{aligned}$$

$$\text{Specificity} = \mathbf{88.24\%}$$

Python Program

Performance Evaluation using Confusion Matrix

TP = 180

TN = 150

FP = 20

FN = 50

Calculations

accuracy = (TP + TN) / (TP + TN + FP + FN)

precision = TP / (TP + FP)

sensitivity = TP / (TP + FN)

specificity = TN / (TN + FP)

Display Results

print("Accuracy =", round(accuracy * 100, 2), "%")

print("Precision =", round(precision * 100, 2), "%")

print("Sensitivity (Recall) =", round(sensitivity * 100, 2), "%")

print("Specificity =", round(specificity * 100, 2), "%")

Output:

```
# Performance Evaluation using Confusion Matrix
```

```
TP = 180
```

```
TN = 150
```

```
FP = 20
```

```
FN = 50
```

```
# Calculations
```

```
accuracy = (TP + TN) / (TP + TN + FP + FN)
```

```
precision = TP / (TP + FP)
```

```
sensitivity = TP / (TP + FN)
```

```
specificity = TN / (TN + FP)
```

```
# Display Results
```

```
print("Accuracy =", round(accuracy * 100, 2), "%")
```

```
print("Precision =", round(precision * 100, 2), "%")
```

```
print("Sensitivity (Recall) =", round(sensitivity * 100, 2), "%")
```

```
print("Specificity =", round(specificity * 100, 2), "%")
```

```
Accuracy = 82.5 %
```

```
Precision = 90.0 %
```

```
Sensitivity (Recall) = 78.26 %
```

```
Specificity = 88.24 %
```

4.A Genetic Algorithm is applied to maximize the function $f(x) = x^2$ using an initial population consisting of the values 2, 4, 6, and 8. Determine the fitness value for each individual in the population and select the best candidates based on their fitness scores. Perform a crossover operation by exchanging the least significant bits of the selected individuals to generate new offspring. Additionally, explain the role of mutation in improving the diversity and effectiveness of the solution in Genetic Algorithms. (5 Marks)

Given:

- Objective Function: $f(x) = x^2$
- Initial Population: 2, 4, 6, 8

Step 1: Calculate the Fitness Value

The fitness function is:

$$\text{Fitness} = f(x) = x^2$$

Individual (x)	Fitness = x^2
2	4
4	16
6	36
8	64

Step 2: Selection

Individuals with the highest fitness values are selected for reproduction.

Selected Individuals:

- 8 (Fitness = 64)
- 6 (Fitness = 36)

These individuals are chosen because they have the highest fitness scores.

Step 3: Crossover

Binary Representation

- 8 = 1000
- 6 = 0110

Exchange the **Least Significant Bit (LSB)** of both parents.

Parent 1: 1000

Parent 2: 0110

After exchanging the last bit:

Offspring 1: 1000 = 8

Offspring 2: 0110 = 6

Since both LSBs are 0, the offspring remain unchanged.

New Offspring:

- Offspring 1 = 8
- Offspring 2 = 6

Python Program

```
# Genetic Algorithm Example
# Initial population
population = [2, 4, 6, 8]
# Calculate fitness
fitness = [x**2 for x in population]
print("Population:", population)
print("Fitness Values:", fitness)

# Select best two individuals
selected = sorted(population, key=lambda x: x**2, reverse=True)[:2]
print("Selected Individuals:", selected)

# Binary representation
p1 = format(selected[0], '04b')
p2 = format(selected[1], '04b')
print("Parent 1:", p1)
print("Parent 2:", p2)

# Crossover by exchanging least significant bit
child1 = p1[:-1] + p2[-1]
child2 = p2[:-1] + p1[-1]
print("Child 1:", child1, "=", int(child1, 2))
print("Child 2:", child2, "=", int(child2, 2))

# Mutation example (flip last bit of child1)
mutated = child1[:-1] + ('1' if child1[-1] == '0' else '0')
print("Mutated Child:", mutated, "=", int(mutated, 2))
```

Output

```
# Genetic Algorithm Example

# Initial population
population = [2, 4, 6, 8]

# Calculate fitness
fitness = [x**2 for x in population]

print("Population:", population)
print("Fitness Values:", fitness)

# Select best two individuals
selected = sorted(population, key=lambda x: x**2, reverse=True)[:2]

print("Selected Individuals:", selected)

# Binary representation
p1 = format(selected[0], '04b')
p2 = format(selected[1], '04b')

print("Parent 1:", p1)
print("Parent 2:", p2)

# Crossover by exchanging least significant bit
child1 = p1[:-1] + p2[-1]
child2 = p2[:-1] + p1[-1]

print("Child 1:", child1, "=", int(child1, 2))
print("Child 2:", child2, "=", int(child2, 2))

# Mutation example (flip last bit of child1)
mutated = child1[:-1] + ('1' if child1[-1] == '0' else '0')

print("Mutated Child:", mutated, "=", int(mutated, 2))

Population: [2, 4, 6, 8]
Fitness Values: [4, 16, 36, 64]
Selected Individuals: [8, 6]
Parent 1: 1000
Parent 2: 0110
Child 1: 1000 = 8
Child 2: 0110 = 6
Mutated Child: 1001 = 9
```

5 .During the training of a deep neural network model, the training and validation accuracies are observed over multiple epochs. The training accuracy increases steadily from 60% to 95%, while the validation accuracy initially increases from 58% to 72% but then decreases to 68% in later epochs. Analyse this trend and identify the issue affecting the model after a certain point in training. Explain why this issue occurs in neural networks and suggest two effective techniques to address it. Also, discuss the significance of validation accuracy in evaluating the performance of a learning model. (5 Marks)

Given:

- Training Accuracy: Increases from 60% to 95%
- Validation Accuracy: Increases from 58% to 72%, then decreases to 68%

Analysis

Initially, both the training and validation accuracies improve, which indicates that the model is learning useful patterns from the training data.

However, after a certain number of epochs:

- The training accuracy continues to increase from 72% to 95%.
- The validation accuracy decreases from 72% to 68%.

This indicates that the model is suffering from Overfitting.

Techniques to Reduce Overfitting

1. Dropout

- Dropout randomly deactivates some neurons during training.
- It prevents the network from depending too much on specific neurons.
- This improves the model's ability to generalize.

2. Early Stopping

- Early stopping monitors the validation accuracy during training.
- Training is stopped when the validation accuracy stops improving or starts decreasing.
- This prevents the model from overfitting the training data.

Conclusion

The decrease in validation accuracy while training accuracy continues to improve indicates overfitting. This happens because the model memorizes the training data instead of learning general patterns. Using Dropout and Early Stopping can effectively reduce overfitting. Validation accuracy is an important metric because it reflects the model's performance on unseen data and helps build a reliable deep learning model.

Python Program

Training and Validation Accuracy Analysis

```
training_accuracy = [60, 70, 80, 90, 95]
validation_accuracy = [58, 65, 72, 70, 68]

print("Epoch\tTraining\tValidation")

for i in range(len(training_accuracy)):
    print(f"{i+1}\t{training_accuracy[i]}\t{validation_accuracy[i]}")

# Detect overfitting
if training_accuracy[-1] > training_accuracy[-2] and validation_accuracy[-1] <
validation_accuracy[-2]:
    print("\nThe model is suffering from Overfitting.")
else:
    print("\nThe model is performing normally.")
```

Output

```
# Training and Validation Accuracy Analysis

training_accuracy = [60, 70, 80, 90, 95]
validation_accuracy = [58, 65, 72, 70, 68]

print("Epoch\tTraining\tValidation")

for i in range(len(training_accuracy)):
    print(f"{i+1}\t{training_accuracy[i]}\t{validation_accuracy[i]}")

# Detect overfitting
if training_accuracy[-1] > training_accuracy[-2] and validation_accuracy[-1] < validation_accuracy[-2]:
    print("\nThe model is suffering from Overfitting.")
else:
    print("\nThe model is performing normally.")
```

Epoch	Training	Validation
1	60%	58%
2	70%	65%
3	80%	72%
4	90%	70%
5	95%	68%

The model is suffering from Overfitting.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand